

EDUCATIONAL PROJECT

Playlist Downloader

Importing exported song lists and retrieving the audio

Flask • yt-dlp • SQLite • Vanilla JavaScript

Every screenshot is the application actually running • 2026-09-17

What we will cover

1. The problem
2. Scope and legal framing
3. Architecture
4. How a download actually runs
5. A real execution, measured
6. What it records
7. Where it breaks
8. What comes next

The problem

A playlist you curated is stuck inside someone else's app

- A curated playlist cannot easily leave the service it lives on.
- Offline situations are ordinary: a flight, a car, a device that does not run the provider's client.
- Existing tools want an account or an API key.

The goal

An exported playlist in, local audio files out — with no sign-in.

The screenshot shows a web interface titled "IMPORT LIST" with a sub-header "Playlist From File". Below this, it says "Export a TXT or CSV song list, then drop it here to build a downloadable playlist." There is a section titled "Need a Spotify list?" which explains that users can export their playlist as TXT or CSV using "Chosic Spotify Playlist Exporter" and then import that file, noting that no Spotify API key is needed. Below this is a "Format" section with two radio buttons: "MP3 Audio" (selected) and "MP4 Video". At the bottom, there is a dashed box containing the text "TXT OR CSV", "Drop your exported playlist here", and "or choose a file from your computer", with a green "Choose File" button. A footer note states: "Accepted formats: one song per line, or CSV with artist/title columns."

Scope, stated plainly

What the tool does

- Issues a search query to a public site
- Retrieves a media stream from a public URL
- Converts and packages the result

Neither step circumvents access controls or DRM.

All demonstration material uses public-domain classical recordings, so nothing here depends on fetching a work in copyright.

What it does not do

- Grant any right to the material retrieved
- Belong on a public host
- Support more than one operator

Responsibility sits with the operator: platform terms of service and copyright law both still apply.

Three ways in

IMPORT LIST

Playlist From File
Export a TXT or CSV song list, then drop it here to build a downloadable playlist.

Need a Spotify list?

Export your playlist as TXT or CSV with [Chosic Spotify Playlist Exporter](#), then import that file here. No Spotify API key is needed.

Format

☒ MP3 Audio ☐ MP4 Video

TXT OR CSV

Drop your exported playlist here
or choose a file from your computer

Choose File

Accepted formats: one song per line, or CSV with artist/title columns.

YOUTUBE

Single Video Download
Download one YouTube video directly as audio or video.

Format

☒ MP3 Audio ☐ MP4 Video

Paste YouTube video URL

Download

Example: <https://www.youtube.com/watch?v=dQw4w9WgXcQ>

YOUTUBE

Playlist Download
Inspect a full YouTube playlist and download the entire set.

Format

☒ MP3 Audio ☐ MP4 Video

Paste YouTube playlist URL

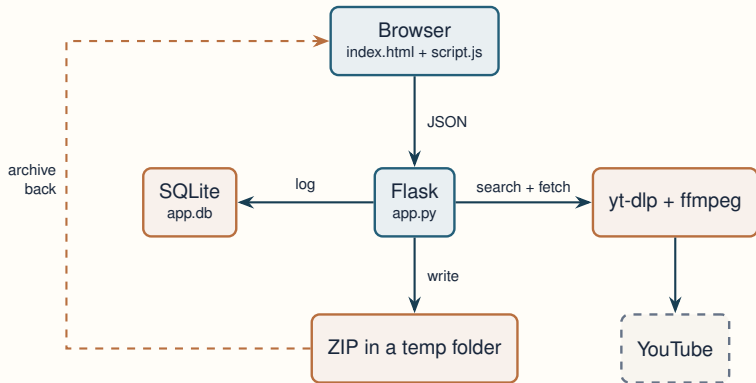
Fetch Playlist

Example: <https://www.youtube.com/playlist?list=PLx0sYbCqOb8TBPRdmBHs5iftv9TPboYG>

A TXT/CSV import, a single YouTube video, or a whole YouTube playlist — each with its own MP3/MP4 selector.

Architecture

One Flask process, no build step, no broker



3041 lines total. The only external process is `yt-dlp`, which calls `ffmpeg` to transcode.

Step 1 — Import, entirely in the browser

- The file is **never uploaded**. Parsing happens in `parseImportedPlaylist()`.
- The parser guesses the shape: CSV with a header, CSV without, or one “Artist - Title” per line.
- Inspecting a playlist therefore costs no network traffic at all.

```
const hasHeader = firstColumns.some(  
  (c) => /track|title|song|artist/.test(c)  
);
```



Step 2 — Matching is one search query

```
def search_youtube_video(song_name, artist):  
    query = f"{artist} {song_name} official audio".strip()  
    return f"ytsearch1:{query}"
```

Why this is good

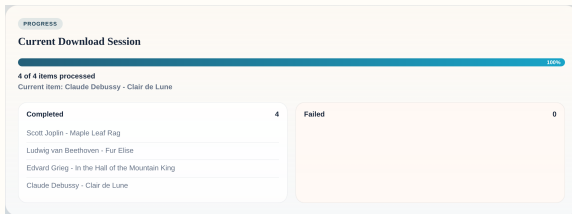
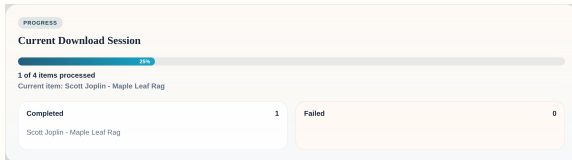
- No API key, no catalogue, no registration
- Search and retrieval collapse into one `yt-dlp` call

What it costs

- You get whatever YouTube ranks first
- A widely compiled piece can resolve to an hour-long mix

Step 3 — Progress, while the work is still running

- One long POST does the work and eventually returns the archive.
- A separate GET `/api/progress` polls once a second, reading the structure the POST is mutating.
- Completed and failed items are kept apart, so partial success stays legible.
- Pausing is cooperative: a flag checked at each track boundary.



A real execution

2026-09-17 — nothing here was simulated

Measured

Tracks requested	4
Completed	4
Failed	0
Wall time	40.5 s
Per track	10.1 s
Archive	19.8 MB
Audio delivered	13:40

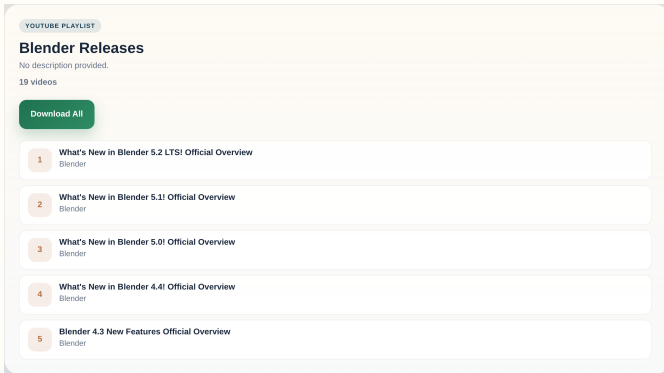
The per-track average includes a deliberate one-second pause between tracks.

What the archive actually held

File	Len	Size	Rate
Claude Debussy - Clair de Lune.mp3	5:11	7.5 MB	192 kbps
Edvard Grieg - In the Hall of the Mountain King.mp3	2:46	4.0 MB	192 kbps
Ludwig van Beethoven - Fur Elise.mp3	2:53	4.2 MB	192 kbps
Scott Joplin - Maple Leaf Rag.mp3	2:50	4.1 MB	192 kbps

Read back off disk with `ffprobe` after the run — not taken from the application's own reporting.

YouTube playlists: look before you fetch



“Blender Releases” resolved to 19 videos with nothing downloaded, using `yt-dlp`’s flat extraction. The “Unknown” channel column is a real defect — see the limitations slide.

What it records



The table has two tabs: 'Activity' (selected) and 'Download History'. It contains four rows of download data, all with a 'Success' status.

TIME	TYPE	ITEM	STATUS	ERROR	IP ADDRESS
9/17/2026, 3:41:00 PM	playlist	Claude Debussy - Clair de Lune	Success	-	127.0.0.1
9/17/2026, 3:40:48 PM	playlist	Edvard Grieg - In the Hall of the Mountain King	Success	-	127.0.0.1
9/17/2026, 3:40:39 PM	playlist	Ludwig van Beethoven - Fur Elise	Success	-	127.0.0.1
9/17/2026, 3:40:31 PM	playlist	Scott Joplin - Maple Leaf Rag	Success	-	127.0.0.1

Two SQLite tables, no user accounts. Behind HTTP Basic auth, compared with `hmac.compare_digest`.
With no password set, the dashboard is closed rather than open.

Where it breaks

Structural

- **Global progress state.** Two simultaneous runs overwrite each other and mix their archives.
- **The working request is the responding request.** Four tracks is 40 s; two hundred would exceed any proxy timeout.
- **Matching has no sanity check.** A wrong-but-successful download is reported as success.

Concrete defects

- Playlist channels all render “Unknown” — flat extraction fills `channel`, not `uploader`.
- Disabling server storage hides the toggle but leaves an empty card.
- `requirements.txt` allows a `yt-dlp` so old that YouTube returns 403 and every track fails.
- `debug=True` on `0.0.0.0` exposes the debugger to the LAN.

What comes next

1. **Confirm the match.** Reject results whose duration is far from the expected track length, or show the candidate first.
2. **Move the work off the request.** A task queue with an identifier removes the global-state and timeout problems together.
3. **Stream progress** with server-sent events instead of a one-second poll.
4. **Tag the output** with ID3 metadata, so the archive is a library rather than a folder of filenames.
5. **Raise the dependency floor** and fail loudly when `yt-dlp` is too old.

In one slide

What was built

A local Flask application that turns an exported playlist file into a ZIP of audio, with visible per-track progress and a small audited log — in about 3 000 lines and with no credentials of any kind.

What was learned

The decision that removes the need for API keys — matching by a single search query — is the same decision that makes matching occasionally wrong. The decision that keeps the code short — progress in a module global — is the same one that confines it to a single user. Useful trades, provided they are stated rather than hidden.

Thank you

Questions?

`python app.py • http://localhost:5000`

MIT licensed • educational use only